

PYTHON SYNTAX CHEATSHEET

◆ CLUSTER 1: CORE LANGUAGE BASICS (FOUNDATION)

This cluster is about **not freezing** on syntax.

1 Python Program Skeleton

```
# single line comment

"""
multi-line
comment / docstring
"""

print("Hello World")
```

Python executes **top to bottom**.

2 Variables & Assignment

```
x = 10
name = "Harish"
price = 99.5
is_active = True
```

- No datatype declaration
- Type decided at runtime

Reassignment allowed:

```
x = 10
x = "ten"
```

3 Multiple Assignment

```
a, b = 10, 20
x = y = z = 0
```

Swap values:

```
a, b = b, a
```

4 Data Types (must-know)

```
int          # 10
float        # 10.5
str          # "abc"
bool         # True / False
None         # None
```

Check type:

```
type(x)
```

5 Input & Output

```
name = input("Enter name: ")
age = int(input("Enter age: "))
```

Output:

```
print(name)
print("Age:", age)
print(f"My name is {name} and age is {age}")
```

6 Type Casting

```
int("10")
float("10.5")
str(100)
bool(1)      # True
bool(0)      # False
```

⚠ Interview tip:

```
bool("False") # True (non-empty string)
```

7 Operators

Arithmetic

```
+ - * / // % **
10 / 3      # 3.333
10 // 3     # 3
10 % 3      # 1
2 ** 3      # 8
```

Comparison

`==` `!=` `>` `<` `>=` `<=`

Returns `True` / `False`

Logical

`and` `or` `not`
`True and False`
`True or False`
`not True`

8 Conditional Statements (VERY IMPORTANT)

if

```
if x > 10:
    print("Greater")
```

if-else

```
if x > 10:
    print("Greater")
else:
    print("Smaller")
```

if-elif-else

```
if x > 10:
    print("Greater")
elif x == 10:
    print("Equal")
else:
    print("Smaller")
```

⚠ Python uses **indentation**, not braces.

9 Truthy & Falsy (INTERVIEW FAVORITE)

Falsy values:

```
0
0.0
""
[]
```

```
{}  
None  
False
```

Everything else → Truthy

Example:

```
if []:  
    print("Won't execute")
```

1 Pass Keyword

```
if x > 10:  
    pass
```

Used as a placeholder.

1 Common Beginner Errors (avoid tomorrow)

 Missing colon:

```
if x > 10    # 
```

 Wrong indentation:

```
if x > 10:  
print("Hi")    # 
```

◆ CLUSTER 2: STRINGS

1 String Creation

```
s1 = "hello"  
s2 = 'hello'  
s3 = """hello  
world"""
```

All are strings.

2 String Indexing (0-based)

```
s = "python"  
s[0]    # 'p'  
s[1]    # 'y'  
s[-1]   # 'n'  
s[-2]   # 'o'
```

⚠ Strings are **immutable**:

```
s[0] = 'P'    # ❌ error
```

3 String Slicing (INTERVIEW FAVORITE)

```
s[start : end : step]  
s = "python"  
  
s[0:4]    # 'pyth'  
s[:4]     # 'pyth'  
s[2:]     # 'thon'  
s[:]      # 'python'  
s[::2]    # 'pto'  
s[::-1]   # 'nohtyp' (reverse)
```

4 Common String Methods (MUST KNOW)

Case conversion

```
s.upper()  
s.lower()  
s.capitalize()  
s.title()
```

Checking methods (return True/False)

```
s.isalpha()      # only letters
s.isdigit()      # only digits
s.isalnum()      # letters + digits
s.isspace()      # whitespace
```

Search & count

```
s.find("py")      # index or -1
s.count("o")      # number of occurrences
```

⚠ Interview tip:

```
s.index("z")      # ❌ error if not found
s.find("z")       # returns -1
```

5 Replace & Strip (AUTOMATION USE)

```
s.replace("py", "PY")
s = "  hello  "
s.strip()         # both sides
s.lstrip()        # left
s.rstrip()        # right
```

6 Split & Join (VERY IMPORTANT)

split()

```
s = "a,b,c"
s.split(",")      # ['a', 'b', 'c']
s = "hello world"
s.split()         # ['hello', 'world']
```

join()

```
lst = ["a", "b", "c"]
",".join(lst)     # 'a,b,c'
```

⚠ join works on **iterables of strings only**

7 String Formatting (INTERVIEW SAFE)

f-string (preferred)

```
name = "Harish"
age = 30
print(f"My name is {name} and age is {age}")
```

format()

```
print("My name is {} and age is {}".format(name, age))
```

% formatting (old)

```
print("My name is %s" % name)
```

8 String Comparison

```
"a" == "a"      # True
"a" < "b"       # True (ASCII)
```

Case-sensitive:

```
"A" != "a"
```

9 Looping Through Strings

```
for ch in "python":
    print(ch)
```

10 Common Interview Traps

Length

```
len("python")    # 6
```

Empty string check

```
if not s:
    print("Empty")
```

Membership

```
"py" in "python"    # True
"x" not in "python" # True
```

Typical Interview Questions (practice mentally)

- Reverse a string

```
s[::-1]
```

- Count vowels

```
for ch in s:
    if ch in "aeiou":
        count += 1
```

- Check palindrome

```
s == s[::-1]
```

Interview-Safe Sentence (memorize)

“Strings in Python are immutable, support slicing, and provide many built-in utility methods.”

◆ CLUSTER 3: LISTS (MOST USED DATA STRUCTURE)

1 List Creation

Syntax	Example	Notes
Empty list	<code>lst = []</code>	Most common
With values	<code>lst = [1, 2, 3]</code>	Mixed types allowed
Constructor	<code>list()</code>	Rarely used

2 List Indexing & Slicing

Operation	Syntax	Output
First element	<code>lst[0]</code>	Value
Last element	<code>lst[-1]</code>	Value
Slice	<code>lst[1:4]</code>	Sublist
Reverse	<code>lst[::-1]</code>	Reversed list

Example:

```
lst = [10, 20, 30, 40]
```

Expression	Result
<code>lst[1:]</code>	<code>[20, 30, 40]</code>
<code>lst[:2]</code>	<code>[10, 20]</code>
<code>lst[::-1]</code>	<code>[40, 30, 20, 10]</code>

3 Modifying Lists (VERY IMPORTANT)

Method	Syntax	Effect
Add one	<code>append(x)</code>	Adds at end
Add many	<code>extend([x, y])</code>	Adds elements
Insert	<code>insert(i, x)</code>	Adds at index
Remove value	<code>remove(x)</code>	Removes first match
Remove index	<code>pop(i)</code>	Removes & returns
Clear	<code>clear()</code>	Empties list

Example:

```
lst.append(50)
lst.pop()
```

List Deletion

Keyword	Syntax	Notes
del	del lst[0]	Removes element
del list	del lst	Deletes list

List Properties

Function	Example	Result
Length	len(lst)	Count
Min	min(lst)	Smallest
Max	max(lst)	Largest
Sum	sum(lst)	Total

Looping Through Lists

Basic loop

```
for x in lst:
    print(x)
```

Index + value

```
for i, v in enumerate(lst):
    print(i, v)
```

List Comprehension (INTERVIEW SAFE)

Purpose	Syntax
Simple	[x for x in lst]
Condition	[x for x in lst if x > 10]
Transform	[x*2 for x in lst]

Example:

```
evens = [x for x in lst if x % 2 == 0]
```

8 Membership Check

Syntax	Result
<code>x in lst</code>	True / False
<code>x not in lst</code>	True / False

9 Sorting Lists

Method	Syntax	Modifies
sort	<code>lst.sort()</code>	Yes
sorted	<code>sorted(lst)</code>	No

Reverse:

```
lst.sort(reverse=True)
```

10 Copying Lists (INTERVIEW TRAP)

Method	Syntax	Safe?
Assignment	<code>b = a</code>	✗ No
Slice	<code>b = a[:]</code>	✓ Yes
copy()	<code>b = a.copy()</code>	✓ Yes

1 1 Nested Lists

```
matrix = [[1,2],[3,4]]
```

Access:

```
matrix[0][1]    # 2
```

1 2 Common Interview Mistakes

Mistake	Why
Modifying list while iterating	Skips items
Using <code>append()</code> instead of <code>extend()</code>	Nested list created
Forgetting list is mutable	Side effects

◆ CLUSTER 4: TUPLES & SETS (TABULAR VIEW)

◆ A. TUPLES

✚ What is a Tuple?

An **ordered, immutable** collection.

✓ Tuple Basics

Feature	Syntax / Notes
Create tuple	<code>t = (1, 2, 3)</code>
Single element tuple	<code>t = (5,)</code> ⚠ comma mandatory
Without brackets	<code>t = 1, 2, 3</code>
Immutable	Cannot modify elements
Ordered	Maintains order
Allows duplicates	Yes

🔍 Tuple Operations

Operation	Example
Indexing	<code>t[0]</code>
Negative index	<code>t[-1]</code>
Slicing	<code>t[1:3]</code>
Length	<code>len(t)</code>
Count	<code>t.count(2)</code>
Index	<code>t.index(3)</code>

⚠ Tuple Interview Traps

Trap	Explanation
<code>t = (1)</code>	✗ Not a tuple
<code>t = (1,)</code>	✓ Correct
Modify element	✗ Error
Faster than list	✓ Yes

When to Use Tuple (Interview Line)

“I use tuples when data should not change, like configuration or fixed records.”

◆ B. SETS

What is a Set?

An **unordered, mutable** collection of **unique elements**.

Set Basics

Feature	Syntax / Notes
Create set	<code>s = {1, 2, 3}</code>
Empty set	<code>s = set()</code> ⚠️ <code>{}</code> is dict
Unordered	No indexing
Unique values	Duplicates removed
Mutable	Can add/remove
No slicing	✗

Set Methods (HIGH FREQUENCY)

Method	Example	Purpose
add	<code>s.add(4)</code>	Add element
update	<code>s.update([5, 6])</code>	Add multiple
remove	<code>s.remove(3)</code>	Error if missing
discard	<code>s.discard(3)</code>	Safe remove
pop	<code>s.pop()</code>	Random remove
clear	<code>s.clear()</code>	Empty set

Set Operations (INTERVIEW FAVORITE)

Operation	Syntax
Union	<code>a b</code>
Intersection	<code>a & b</code>
Difference	<code>a - b</code>

Operation Syntax

Symmetric diff $a \mathbin{\Delta} b$

Set Use Cases

Scenario	Why Set
----------	---------

Remove duplicates	Ensures uniqueness
-------------------	--------------------

Membership check	Faster than list
------------------	------------------

Common elements	Intersection
-----------------	--------------

Set Interview Traps

Trap	Explanation
Indexing	✗ Not allowed
Order expected	✗ No guarantee
{ }	Creates dict, not set

◆ C. LIST vs TUPLE vs SET (VERY IMPORTANT)

Feature	List	Tuple	Set
Ordered	✓	✓	✗
Mutable	✓	✗	✓
Duplicates	✓	✓	✗
Indexing	✓	✓	✗
Performance	Medium	Fast	Fastest lookup
Use case	General data	Fixed data	Unique data

◆ CLUSTER 5: DICTIONARIES (VERY HIGH INTERVIEW FREQUENCY)

Dictionaries are **key–value stores**. Extremely common in automation, APIs, JSON, configs.

1 Dictionary Creation

Syntax	Example	Notes
Empty dict	<code>d = {}</code>	Most common
With values	<code>d = {"a": 1, "b": 2}</code>	Keys must be unique
Constructor	<code>dict(a=1, b=2)</code>	Keys must be strings

2 Accessing Values

Operation	Syntax	Result
By key	<code>d["a"]</code>	Value
Safe access	<code>d.get("a")</code>	Value / None
Default	<code>d.get("x", 0)</code>	0 if key missing

⚠ Interview trap:

```
d["x"] # ❌ KeyError if not exists
```

3 Adding & Updating Elements

Operation	Syntax	Effect
Add new	<code>d["c"] = 3</code>	Adds key
Update	<code>d["a"] = 10</code>	Updates value
Update many	<code>d.update({"a": 5})</code>	Bulk update

4 Removing Elements

Method	Syntax	Notes
pop	<code>d.pop("a")</code>	Removes key
popitem	<code>d.popitem()</code>	Removes last item
del	<code>del d["a"]</code>	Removes key

Method	Syntax	Notes
clear	<code>d.clear()</code>	Empties dict

5 Dictionary Properties

Function	Example	Result
Length	<code>len(d)</code>	Count of keys
Keys	<code>d.keys()</code>	dict_keys
Values	<code>d.values()</code>	dict_values
Items	<code>d.items()</code>	(key, value) pairs

6 Looping Through Dictionaries (IMPORTANT)

Loop keys (default)

```
for k in d:
    print(k)
```

Loop values

```
for v in d.values():
    print(v)
```

Loop key & value

```
for k, v in d.items():
    print(k, v)
```

7 Dictionary Comprehension (INTERVIEW FAVORITE)

Purpose	Syntax
Simple	<code>{k: v for k, v in d.items()}</code>
Condition	<code>{k: v for k, v in d.items() if v > 10}</code>
Transform	<code>{k: v*2 for k, v in d.items()}</code>

Example:

```
squared = {x: x*x for x in range(5)}
```

8 Checking Key Existence

Syntax	Result
"a" in d	True / False
"a" not in d	True / False

Preferred over try/except for existence check.

9 Nested Dictionaries (VERY COMMON)

```
user = {
    "name": "Harish",
    "address": {
        "city": "Chennai",
        "pin": 600001
    }
}
```

Access:

```
user["address"]["city"]
```

10 Dictionary Copying (INTERVIEW TRAP)

Method	Syntax	Safe?
Assignment	b = a	✗ No
copy	b = a.copy()	⚠ Shallow
deepcopy	copy.deepcopy(a)	✓ Yes

1 1 Common Interview Use-Cases

Scenario	Why dict
JSON / API response	Natural mapping
Counting frequency	Key → count
Configuration	Key-value pairs
Test data	Structured access

Example (frequency count):

```
freq = {}
for ch in "hello":
    freq[ch] = freq.get(ch, 0) + 1
```

1 2 Common Mistakes

Mistake	Why
Using list as key	Unhashable
Accessing missing key	KeyError
Modifying dict while iterating	Runtime error

◆ CLUSTER 6: LOOPS & CONTROL FLOW (CRITICAL)

Loops are everywhere: data processing, automation, validations.

1 **for** Loop Basics

Use	Syntax	Notes
Loop list	<code>for x in lst:</code>	Most common
Loop string	<code>for ch in s:</code>	Char by char
Loop dict keys	<code>for k in d:</code>	Default
Loop dict items	<code>for k,v in d.items():</code>	Key + value

Example:

```
for x in [1, 2, 3]:  
    print(x)
```

2 **range()** Function (VERY IMPORTANT)

Syntax	Output
<code>range(5)</code>	0–4
<code>range(1, 5)</code>	1–4
<code>range(1, 10, 2)</code>	1,3,5,7,9

Example:

```
for i in range(3):  
    print(i)
```

3 **while** Loop

Use	Syntax
Condition loop	<code>while condition:</code>

Example:

```
i = 0  
while i < 3:  
    print(i)  
    i += 1
```

⚠ Interview trap: infinite loop if no update.

4 **break** Statement

Purpose	Effect
Exit loop	Stops immediately

Example:

```
for x in lst:
    if x == 5:
        break
```

5 **continue** Statement

Purpose	Effect
Skip iteration	Moves to next loop

Example:

```
for x in lst:
    if x % 2 == 0:
        continue
    print(x)
```

6 **pass** Statement

Purpose	Use
Placeholder	Empty block

Example:

```
for x in lst:
    pass
```

7 **else** with Loops (INTERVIEW FAVORITE)

for-else

```
for x in lst:
    if x == 5:
        break
```

```
else:
    print("Not found")
```

✓ else runs **only if loop does NOT break**

while-else

```
while i < 5:
    if i == 3:
        break
    i += 1
else:
    print("Completed")
```

8 Nested Loops

Structure **Example**
for inside for Matrix traversal

Example:

```
for i in range(2):
    for j in range(2):
        print(i, j)
```

9 Looping with `enumerate()`

Use **Syntax**
Index + value `enumerate(iterable)`

Example:

```
for i, v in enumerate(lst):
    print(i, v)
```

10 Looping with `zip()`

Use **Syntax**
Parallel iteration `zip(a, b)`

Example:

```
for x, y in zip([1,2], [3,4]):
```

```
print(x, y)
```

1 1 Common Loop Mistakes

Mistake	Why
Modifying list in loop	Skips items
Missing increment in while	Infinite loop
Using break incorrectly	else never runs

1 2 Performance Tip (INTERVIEW SAFE)

- Prefer `for` over `while`
 - Avoid deep nesting
 - Use comprehension when readable
-

◆ CLUSTER 7: FUNCTIONS (VERY HIGH INTERVIEW WEIGHT)

Functions test **logic clarity**, **parameter handling**, and **Python fluency**.

1 Function Definition

Purpose	Syntax	Notes
Define function	<code>def func():</code>	Must use <code>def</code>
Function body	Indented block	Mandatory
Call function	<code>func()</code>	Executes

Example:

```
def greet():  
    print("Hello")  
  
greet()
```

2 Function with Parameters

Type	Syntax	Example
Single param	<code>def f(x):</code>	<code>f(10)</code>
Multiple params	<code>def f(a,b):</code>	<code>f(1,2)</code>

Example:

```
def add(a, b):  
    print(a + b)
```

3 Return Statement (VERY IMPORTANT)

Behavior	Example	Result
Return value	<code>return x</code>	Sends value
No return	No <code>return</code>	Returns <code>None</code>

Example:

```
def add(a, b):  
    return a + b
```

⚠ Interview trap:

```
print(add(1,2))    # prints result
```

4 Default Arguments

Feature	Syntax	Notes
Default value	<code>def f(x=10):</code>	Optional param
Override	<code>f(20)</code>	Replaces default

Example:

```
def greet(name="Guest"):  
    print(name)
```

5 Keyword Arguments

Use	Syntax
Named call	<code>f(a=10, b=20)</code>

Order does not matter.

Example:

```
add(b=2, a=1)
```

6 Positional vs Keyword (INTERVIEW FAVORITE)

Type	Rule
Positional	Order matters
Keyword	Order doesn't matter
Rule	Positional first

✗ Invalid:

```
f(a=10, 20)
```

7 `*args` (Variable Positional Arguments)

Purpose	Syntax	Notes
Multiple values	<code>*args</code>	Tuple

Example:

```
def add(*args):  
    return sum(args)
```

Call:

```
add(1,2,3)
```

8 ****kwargs (Variable Keyword Arguments)**

Purpose	Syntax	Notes
---------	--------	-------

Key-value args	**kwargs	Dict
----------------	----------	------

Example:

```
def show(**kwargs):  
    print(kwargs)
```

Call:

```
show(name="Harish", age=30)
```

9 **Order of Parameters (INTERVIEW TRAP)**

Correct order:

```
def func(positional, default=1, *args, **kwargs):  
    pass
```

✗ Wrong order causes error.

10 **Lambda Functions (ONE-LINERS)**

Use	Syntax
-----	--------

Anonymous func	lambda x: x*2
----------------	---------------

Example:

```
square = lambda x: x*x
```

Used with:

```
map(), filter(), sorted()
```

1 1 `map()`, `filter()` (BASIC AWARENESS)

Function	Purpose	Example
<code>map</code>	Transform	<code>map(f, lst)</code>
<code>filter</code>	Condition	<code>filter(f, lst)</code>

Example:

```
list(map(lambda x: x*2, [1,2,3]))
```

1 2 Docstrings

Use	Syntax
Documentation	<code>"""text"""</code>

Example:

```
def add(a,b):  
    """Returns sum of two numbers"""  
    return a+b
```

1 3 Scope (INTERVIEW FAVORITE)

Scope	Meaning
Local	Inside function
Global	Outside function

Example:

```
x = 10  
def f():  
    x = 5
```

Use `global`:

```
global x
```

⚠️ Avoid using `global` in real code.

1 4 Common Function Mistakes

Mistake	Why
Forgetting return	Returns None
Mutable default args	Unexpected behavior
Overusing lambda	Reduces readability

Mutable default trap:

```
def f(x=[]): # ❌
```

◆ PHASE 8: OBJECT-ORIENTED PROGRAMMING (EXPANDED WITH DEFINITIONS)

◆ 8.1 What is OOP? (Core Definition)

Term	Definition
OOP	A programming paradigm based on objects and classes
Object	Real-world entity with data (state) and behavior
Goal	Modularity, reusability, maintainability

✚ Interview-safe definition:

“OOP is a paradigm that models real-world entities using objects that encapsulate data and behavior.”

◆ 8.2 Class (Blueprint)

Aspect	Explanation
Class	Blueprint or template for creating objects
Contains	Variables (attributes) + methods (behavior)
Keyword	<code>class</code>
	<pre>class Employee: pass</pre>

✚ Interview line:

“A class defines what an object will look like and what it can do.”

◆ 8.3 Object (Instance)

Aspect	Explanation
Object	Instance of a class
Memory	Allocated when object is created
Purpose	Access class members
	<pre>emp = Employee()</pre>

✚ Interview line:

“An object is a runtime instance of a class.”

◆ 8.4 Constructor (`__init__`)

Aspect	Explanation
Constructor	Special method to initialize object
Auto-call	Yes, during object creation
Purpose	Assign initial values
<pre>class Employee: def __init__(self, name): self.name = name</pre>	

✦ Interview line:

“`__init__` initializes the state of an object.”

◆ 8.5 `self` Keyword (VERY IMPORTANT)

Aspect	Explanation
<code>self</code>	Reference to current object
Mandatory	First parameter of instance methods
Use	Access instance variables & methods
<pre>self.name</pre>	

✦ Interview trap:

`self` is not a keyword, but a convention.

◆ 8.6 Instance Variables vs Class Variables

Instance Variables

Feature	Explanation
Scope	Object-specific
Defined	Inside <code>__init__</code>
Changes	Affect only that object
<pre>self.salary = 50000</pre>	

Class Variables

Feature	Explanation
Scope	Shared across objects
Defined	Inside class, outside methods
Changes	Affect all objects

```
company = "ABC Corp"
```

✦ Interview line:

“Instance variables are object-specific, class variables are shared.”

◆ 8.7 Instance Methods

Aspect	Explanation
Method	Function inside a class
First param	<code>self</code>
Purpose	Define object behavior

```
def get_details(self):  
    return self.name
```

◆ 8.8 Encapsulation (Data Hiding)

Aspect	Explanation
Encapsulation	Wrapping data and methods together
Goal	Protect data from direct access
Technique	Private variables

```
self.__balance = 0
```

✦ Interview line:

“Encapsulation protects object integrity by restricting direct access.”

◆ 8.9 Getters & Setters

Aspect	Explanation
Getter	Read private data
Setter	Validate & update data
Benefit	Controlled access

```
def set_balance(self, amt):  
    if amt > 0:  
        self.__balance = amt
```

◆ 8.10 Inheritance (IS-A Relationship)

Aspect	Explanation
Inheritance	Child acquires parent properties
Purpose	Code reuse
Relationship	IS-A

```
class FullTimeEmployee(Employee):  
    pass
```

✦ Interview line:

“Inheritance models an IS-A relationship.”

◆ 8.11 Method Overriding

Aspect	Explanation
Overriding	Child redefines parent method
Runtime	Decided at runtime
Enables	Polymorphism

```
def calculate_salary(self):  
    pass
```

◆ 8.12 `super()` Keyword

Aspect	Explanation
<code>super()</code>	Access parent class methods
Use	Avoid duplication
Common In	Constructors

```
super().__init__(name)
```

◆ 8.13 Polymorphism (INTERVIEW GOLD)

Aspect	Explanation
Polymorphism	One interface, many forms
Achieved via	Method overriding

Aspect	Explanation
Benefit	Loose coupling
<code>emp.calculate_salary()</code>	

✦ Interview line:

“Polymorphism allows the same method call to behave differently at runtime.”

◆ 8.14 Abstraction (Contracts)

Aspect	Explanation
Abstraction	Hiding implementation details
Goal	Enforce rules
Tool	Abstract Base Classes
<code>from abc import ABC, abstractmethod</code>	

✦ Interview line:

“Abstraction enforces a contract without implementation.”

◆ 8.15 Abstract Methods

Aspect	Explanation
Abstract method	Method without body
Rule	Child must implement
When to use	No sensible default
<code>@abstractmethod</code> <code>def calculate_salary(self):</code> <code> pass</code>	

◆ 8.16 Composition (HAS-A Relationship)

Aspect	Explanation
Composition	Object inside another object
Relationship	HAS-A
Benefit	Flexibility
<code>self.engine = Engine()</code>	

✦ Interview line:

“Composition is preferred over inheritance for flexibility.”

◆ 8.17 `__str__` Method

Aspect	Explanation
--------	-------------

Purpose	String representation
---------	-----------------------

Trigger	<code>print(object)</code>
---------	----------------------------

def	<code>__str__(self):</code> <code> return self.name</code>
-----	--

◆ 8.18 Common OOP Interview Mistakes

Mistake	Why it's bad
Overusing inheritance	Tight coupling
Everything abstract	Overengineering
Forgetting <code>self</code>	Runtime error
No justification	Weak design

◆ PHASE 9: EXCEPTION HANDLING (EXPANDED WITH DEFINITIONS)

This phase is **very important** for senior interviews because it shows how you think about **failures**, not just success paths.

◆ 9.1 What is an Exception? (Core Definition)

Term	Definition
Exception	An error that occurs during program execution
Nature	Runtime error
Effect	Disrupts normal program flow

✦ Interview-safe definition:

“An exception is a runtime error that interrupts the normal flow of a program.”

◆ 9.2 Why Exception Handling is Needed

Reason	Explanation
Prevent crash	Program continues gracefully
Controlled failure	Known error paths
Debugging	Clear error context
Reliability	Production safety

✦ Interview line:

“Exception handling allows graceful degradation instead of abrupt termination.”

◆ 9.3 `try / except` Block

Block	Purpose
<code>try</code>	Code that may fail
<code>except</code>	Handle exception
Flow	If error → <code>except</code> runs

```
try:  
    x = int("abc")  
except ValueError:
```

```
print("Invalid number")
```

⚠️ Code after error in `try` does NOT execute.

◆ 9.4 Catching Specific Exceptions (IMPORTANT)

Exception	When it occurs
<code>ValueError</code>	Invalid type conversion
<code>TypeError</code>	Wrong data type
<code>KeyError</code>	Missing dict key
<code>IndexError</code>	List index out of range
<code>ZeroDivisionError</code>	Divide by zero
<code>FileNotFoundError</code>	Missing file

✳️ Interview rule:

Catch specific exceptions, not generic ones.

◆ 9.5 Multiple `except` Blocks

```
try:
    x = int(input())
except ValueError:
    print("Value error")
except ZeroDivisionError:
    print("Divide by zero")
```

Order matters:

- Specific first
 - Generic last
-

◆ 9.6 Generic `except` (Use Carefully)

```
except Exception as e:
    print(e)
```

⚠️ Interview warning:

Avoid blanket exception handling unless logging.

◆ 9.7 `else` with `try-except` (INTERVIEW FAVORITE)

Block **When it runs**

```
else    If no exception occurs
try:
    x = int("10")
except ValueError:
    print("Error")
else:
    print("Success")
```

◆ 9.8 `finally` Block (VERY IMPORTANT)

Aspect **Explanation**

Purpose Always executes

Use Cleanup

Examples Close file, quit browser

```
try:
    f = open("data.txt")
finally:
    f.close()
```

✦ Automation tie-in:

`finally` is like teardown.

◆ 9.9 Raising Exceptions (`raise`)

Use **Explanation**

Manual error Force exception

Validation Input checks

```
if amount <= 0:
    raise ValueError("Invalid amount")
```

✦ Interview line:

“I raise exceptions to enforce business rules.”

◆ 9.10 Custom Exceptions

Aspect	Explanation
Custom error	Domain-specific
Inherits	Exception
<pre>class InsufficientBalance(Exception): pass</pre>	

Usage:

```
raise InsufficientBalance("Low balance")
```

◆ 9.11 Exception Propagation

Concept	Explanation
Propagation	Unhandled exception moves up
Caller	Can catch

✦ Interview line:

“Exceptions propagate up the call stack if not handled.”

◆ 9.12 When NOT to Use try–except (IMPORTANT)

Scenario	Reason
Normal control flow	Bad practice
Expected condition	Use if-else
Silent failure	Hides bugs

◆ 9.13 Logging vs Printing Exceptions

Aspect	Print Logging	
Production	✗	✓
Debugging	⚠	✓
Traceability	✗	✓

✦ Interview-safe line:

“I log exceptions instead of printing them in production code.”

◆ 9.14 Common Exception Handling Mistakes

Mistake	Why it's bad
Catching Exception everywhere	Hides bugs
Ignoring exceptions	Silent failure
No cleanup	Resource leaks
Using exceptions for flow	Bad design

◆ 9.15 Automation Context (VERY RELEVANT FOR YOU)

Scenario	Handling
Selenium failure	try-except + screenshot
API failure	raise + log
Test cleanup	finally / teardown

PHASE 9 INTERVIEW ONE-LINERS (MEMORIZE)

- “Exceptions are runtime errors.”
 - “I catch specific exceptions first.”
 - “`finally` is used for cleanup.”
 - “I raise exceptions to enforce business rules.”
 - “I don’t use exceptions for normal flow.”
-

◆ PHASE 10: FILE HANDLING (EXPANDED WITH DEFINITIONS)

This phase is **very common** in interviews and automation projects (logs, reports, CSV, configs).

◆ 10.1 What is File Handling? (Core Definition)

Term	Definition
File Handling	Reading data from and writing data to files
Purpose	Persist data beyond program execution
Medium	Text files, CSV, logs, configs

✦ Interview-safe definition:

“File handling allows a program to read from and write to files for persistent storage.”

◆ 10.2 Types of Files (Awareness)

File Type	Description
Text file	Human-readable (.txt, .log)
Binary file	Non-readable (.png, .exe)
Structured	CSV, JSON, XML

Most interviews focus on **text + CSV**.

◆ 10.3 Opening a File (`open()`)

Aspect	Explanation
Function	<code>open()</code>
Returns	File object
Syntax	<code>open(filename, mode)</code> <code>f = open("data.txt", "r")</code>

◆ 10.4 File Modes (VERY IMPORTANT)

Mode	Meaning
r	Read (default)
w	Write (overwrite)
a	Append
x	Create new file
rb	Read binary
wb	Write binary

📌 Interview trap:

w mode creates file if not exists and overwrites if exists

◆ 10.5 Reading from Files

Read entire file

```
f.read()
```

Read one line

```
f.readline()
```

Read all lines

```
f.readlines()
```

Method	Returns
<code>read()</code>	String
<code>readline()</code>	One line
<code>readlines()</code>	List of lines

◆ 10.6 Writing to Files

```
f.write("Hello\n")
```

⚠️ `write()` does NOT add newline automatically.

Multiple lines:

```
f.writelines(["a\n", "b\n"])
```

◆ 10.7 Closing Files (CRITICAL)

Aspect	Explanation
--------	-------------

<code>close()</code>	Releases resources
----------------------	--------------------

Risk	File lock, memory leak
------	------------------------

```
f.close()
```

📌 Interview line:

“Files must always be closed after use.”

◆ 10.8 `with` Statement (BEST PRACTICE)

Feature	Explanation
---------	-------------

Auto close	Yes
------------	-----

Safer	Yes
-------	-----

Recommended Always

```
with open("data.txt", "r") as f:  
    data = f.read()
```

📌 Interview-safe line:

“I use `with` to ensure files are closed automatically.”

◆ 10.9 File Pointer (`seek`, `tell`)

Method	Purpose
--------	---------

<code>tell()</code>	Current position
---------------------	------------------

<code>seek(n)</code>	Move pointer
----------------------	--------------

```
f.seek(0)
```

Used rarely, but good awareness.

◆ 10.10 Reading Files Line by Line (MEMORY SAFE)

```
with open("data.txt") as f:  
    for line in f:  
        print(line)
```

📌 Interview line:

“Iterating line by line is memory efficient.”

◆ 10.11 File Existence Check

```
import os

os.path.exists("data.txt")
```

Other useful checks:

```
os.path.isfile()
os.path.isdir()
```

◆ 10.12 Deleting Files

```
import os
os.remove("data.txt")
```

⚠ File must exist, else `FileNotFoundError`.

◆ 10.13 Working with CSV Files (HIGHLY PRACTICAL)

Reading CSV

```
import csv

with open("data.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Writing CSV

```
with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "age"])
```

📌 Interview tie-in:

Used for test data & reports.

◆ 10.14 Exception Handling with Files

```
try:
    with open("data.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("File missing")
```

✦ Best practice:

Combine with `+ try-except`.

◆ 10.15 Common File Handling Mistakes

Mistake	Why it's bad
Forgetting close	Resource leak
Using <code>w</code> accidentally	Data loss
Reading large files at once	Memory issue
Hard-coded paths	Non-portable

◆ 10.16 Automation Context (VERY IMPORTANT FOR YOU)

Scenario	Usage
Logs	Read / write
Test data	CSV / JSON
Reports	Write HTML / text
Screenshots	Binary write
